



MMA Companion

Computer Science NEA

Thomas Jubb

Contents

Effectiveness Of the Programming Language and Tools	2
Compare and Contrast with Similar Commercial Systems	3
Topology.....	3
Official UFC Website	3
FightMatrix.....	4
Identify Good Features and Shortcomings	5
Significant Potential Improvements	5
Own Strengths and Weaknesses During Design and Development	6
Future Changes to Approach to Avoid Problems.....	6

Effectiveness Of the Programming Language and Tools

The programming language I used for this project was Python. I used python because it is very easy to read compared to other programming languages and the other logic and arithmetic rules it sets are very easy to understand making the learning curve relatively lower than other programming languages. Python had libraries for everything I needed such as QT designer integration, SQL database management and statistical chart making. However, Python can be slower than the other languages I could've used. Also, Python programs are way harder to distribute as an application due to them being harder to compile so in a next project if I had to distribute it another programming language would be more suitable.

I used a wide variety of tools for the development of my project. Firstly, for the design stage I used Lucidchart to create the user/admin use cycle flowchart. I think that this tool is very simple to use with features like an object library and arrow snapping. In my next project, I will use this tool to design a flowchart and maybe even the screen concepts mainly because of how quick and simple it makes the design process. However, if the project design is very intricate I will have to pay a subscription fee as Lucidchart has a set amount of charts you can make in the program with the free version.

Secondly, for the UI design I used QT designer. QT designer is advantageous because it has a GUI to create the UI as opposed to code like you would use with tkinter or flask. This makes development faster by reducing the time spent on writing code. This also helps with organisation by making the separation between UI code and program code. However, some advanced components of the UI still require me to do some coding, which can weigh down on this specific advantage. Also in some cases, it also can be more convenient for the user to have a web application as opposed to an application that they would have to download. I think that given another project where a large userbase intends to use it, the time taken to learn something like flask would be worth the benefits of how easy a web GUI is for users to access.

Thirdly, for projects like these the tool you will use most is probably going to be the IDE. In my case the IDE I used was Visual Studio Code. Visual Studio Code is advantageous because of the wide range of features it has. The one that has helped me the most is the built-in debugger. The built-in debugger lets me spot minor errors before I run the program, saving me valuable time. Another advantage that the IDE provides is the rollback feature it provides. This is very helpful because it means that if I make a detrimental mistake I can just roll back the version and my code won't be damaged. However, a lot of the advantages of the program rely on the many add-ons the IDE provides. This becomes a problem if I had a project that I had to provide long-term maintenance for because these add-ons can become deprecated over time. Also, some of these add-ons can conflict which can cause major problems in a project. I think I would use this IDE in my future projects, keeping in mind the risks associated with it.

Finally, a feature I used quite prevalently in my development process was version management software. The program I used for this was GitHub. Like VS Code, GitHub provides rollback capabilities. Although I didn't use them in this project, if I were to switch to an IDE which didn't have rollback capabilities in a future project I could use GitHub instead. GitHub also stores the code online meaning that you can access it from different devices. This was very useful to me when I needed to switch the code to-and-from my personal laptop, computer and the school computers. GitHub provides labelled changes which allows me to track my development and set milestones in my progress. However, you could argue that a local backup like a USB stick could've provided what I needed for the project and GitHub is overkill. Also, conflicts between versions can happen if I'm not

consistent with the backups I make. In a future project, I'll most probably use this tool as the capabilities it has makes it very useful.

Compare and Contrast with Similar Commercial Systems

Tapology

Tapology is a large MMA statistics website that lets you track fighters, fights, rankings and predictions.

Similarities

Both systems store fighter records and fight results. The fighter records and results can be viewed by the user and are also available as statistics too. Both systems provide rankings or statistical comparison features.

Tapology Advantages

Tapology features a very big dataset due to how popular it is. It also has community prediction and discussion features with a professional web UI that's easy to access from multiple devices unlike my system which is only accessible through a windows app.

Advantages of my system

My system has a different framework to Tapology as my system is built off algorithm-based statistics to determine rankings rather than fan opinion which is more of the way of Tapology. My system is also focused more from data analysis and predictions rather than community features. This can be seen by the odds calculator feature that analyses fighter data.

Official UFC Website

The UFC has its own official statistics website where it shows the statistics for all of the fighters that are in the UFC.

Similarities

Both systems let their users view fighter profiles. From these profiles you can see fighter records and stats in charts. Both systems also feature a ranked leaderboard.

UFC Website Advantages

The UFC website has a professional web design like tapology which again makes it easier to access. It also includes videos, interviews and news about the UFC. The UFC website also features the official rankings and upcoming fights. Like tapology it's a very large dataset and it's also subject to constant updates

My System Advantages

My system allows for detailed statistical analysis and prediction but it also has custom searching features which the UFC website doesn't provide.

FightMatrix

FightMatrix is an MMA ranking website that uses algorithms to rank fighters.

Similarities

Unlike the other two MMA websites I just analysed, FightMatrix is a leaderboard system that uses data algorithms instead of opinion polling to figure out fighter rankings. They both focus on data analysis rather than entertainment. They also both work on a table based platform.

FightMatrix Analysis

FightMatrix has a complex ranking algorithm that uses large historic datasets. The algorithms for FightMatrix are so well made that they are well known and referenced widely in MMA analysis.

My System's Advantages

My system has an interface to view individual fighter stats instead of just leaderboard rankings. My system also includes more ways of interpreting the data instead of just a leaderboard by showing fighter data in ways like pie charts and notable fights lists.

Identify Good Features and Shortcomings

I would like to firstly go through all the good features I believe that my system has. I think that the navigation routes in my system are done well. I managed to follow the “three click rule” mantra that is seen in professional design where every page is available in less than three clicks. The UI is very simple to use in my opinion because each input is labelled and each rejection is clearly explained on the admin side, and on the user side each piece of info you see is viewed predictably and is also clearly labelled like the admin side.

Secondly, I think that my data storage was done well. My data was organized in a form which makes all operations such as retrieval, deletion and insertion simple and without as much risk of data issues such as redundancy. I think that I also treat the handling of personal data quite well. The most compromising data to the user, the password (because most people reuse passwords) is secured with not only a hashing algorithm but also with a salting algorithm to add an extra layer of security.

Thirdly, I think that my program gets info across to the user very well. It does this by letting the user go through the info using things such as advanced searching algorithms, filterable tables and when the user finds the data they want it represents it using statistical overviews such as pie charts, streaks and notable fights.

Fourthly, with use of object-oriented programming I have made the program’s code very modular, making it easy to maintain as features are clearly separatable.

However, there also a few issues that my system also has. The UI in my program is quite basic compared to other applications like tapology as flask would’ve been a better choice due to its ease of use, especially with the use case of a regular user.

Also, due to using inefficient solutions to some tasks, if the dataset were to be very large the performance could be slow with some tasks. An example of this would be the table filtering as if the table was very large it would take a long time to sort as QT doesn’t use the most efficient sorting algorithm.

Significant Potential Improvements

Firstly, I have already mentioned this a few times in the document but personally the most important improvement I could make to my program is by changing my UI platform from PyQt to a Flask web application. This is because a flask application is so much more professional than a QT designer program if done properly primarily because it is a familiar concept to most users, being a web app as opposed to a whole program just for MMA stats.

Secondly, I would add ways to make data input easier for admins. A really good way I could improve data input is by adding a spreadsheet import option which can let an admin simultaneously add a large amount of MMA data to the database.

Thirdly, I think that I could expand the odds calculator page and rework it into a “Fighter Comparison” page where both fighters have their stats side by side and you can also see common fighters they’ve both fought against and how either one did against them. I could also add things to the odds calculator page such as the ranking changes if either fighter were to win the fight.

Fourthly, I think that I could expand the options panel to add a user management panel where you can manage users by doing things like removing them, promoting them to and demoting them from admin. On the admin panel, I could also add something such as an audit log where you can track what users made what edits.

Finally, I could add a global search feature. This feature would let you search through all fights, events and fighters. For the events and fights you would be given an overview such as algorithm predictions vs reality to see if the fight was an upset for one of the fighters or not, or a method counter if the user wants to see how often fights in an event ended in knockouts.

Own Strengths and Weaknesses During Design and Development

Firstly, one strength that I had during my project is that my development process was very planned out. I sorted each process by importance and developed them in that order which means that my programming went quite fluidly and because I got the most important things done first the strategy I used was quite forgiving for me.

Secondly, I think that I demonstrated good problem-solving skills during my debugging process. This can be shown by my developmental testing process where I identified a bug, found the exact error in my code using the info I got from error messages and by manually tracing the code, and then corrected the issues with my code.

Thirdly, I think that my progress was quite consistent. I was able to go through the coursework process sequentially and not go back on myself continuously. The way I organised my design documents was done well as I was easily able to build my program on top of my designs.

Fourthly, I think my research was very successful as I was firstly able to identify common features that users will want to see in an MMA leaderboard program and I was able to identify key issues with different MMA applications such as the poor searching algorithms used sometimes.

However, I think that for all the strengths that I had when developing this project, I did have quite a few issues that I had to come to face with during it as well. Firstly, I slightly underestimated the amount of time that I would have to spend debugging my code. I personally think that this is an effect of never having a project of this size before as well, so I tried to tackle this project by coding the project as fast as possible. Because of this I saw a lot of the code as done even though the debugging process which takes longer has yet to even happen yet.

Secondly, I had to simplify some features because I had time constraints. All if not most of the simplified features weren't features that were crucial to the program but most of them still would've made the user experience better if they were still included.

Finally, I think that earlier on with parts of the documentation like my discussion I could've been more detailed. I was able to get into the workflow later on but I think in the beginning part of the coursework I was overtly focused on organising everything and didn't focus on the actual documentation itself.

Future Changes to Approach to Avoid Problems

Firstly, I think that I should've spent more time on my initial design and requirements because I have learnt that for myself that the time spent on the design and making it as intricate as possible would've saved me more time than doing a sketchy design and going ahead with the development as quickly as possible. I should've also analysed my requirements more carefully and more importantly analysed where the features I wanted to add were used and how they were used to see if the feature I wanted to add onto the requirements were even worth it or not.

Secondly, I think I should've been way more organized from the get-go and made something like a Gantt chart and followed it religiously. If I do this in the future I can organise the Gantt chart to have a few weeks gap at the end of the project to look for feedback and refine my project.

Thirdly, I think that I could've organised my development more. What I think I could do for a large programming project in the future that is object-oriented programming dominant like this one is that I could've created a tree like chart which has the most crucial features/the parent objects as the roots with their child objects branching off them

Fourthly, I think I should've been way more iterative with my approach. I think that I used more of a waterfall approach for my whole development when I really should've created small drafts of the program or sub-programs, went to someone for feedback and then applied those pieces of feedback to the system.